JEDAI FLOW

The AI Automation Mastery Guide

The definitive technical reference for building intelligent, scalable automation systems that run your business on autopilot.

ADVANCED EDITION - 2026

11 CHAPTERS 50+ PAGES 20 PRODUCTION PROMPTS REAL CODE EXAMPLES

Table of Contents

00 Introduction: Beyond Basic Automation

01 Multi-Step Workflow Architecture

02 AI-Powered Automations

03 Database-Driven Automations

04 API Integrations Deep Dive

05 Advanced Communication Flows

06 Error Handling & Monitoring

07 White-Label & Client Management

08 Scaling Infrastructure

09 Building AI Agents

++ Bonus: Advanced Prompt Library (20 Prompts)

INTRODUCTION

Beyond Basic Automation

Why most automations fail, and the architectural mindset that separates amateur workflows from production-grade systems.

The Automation Maturity Model

Most businesses that adopt automation tools like Make.com, Zapier, or n8n plateau at Level 1 -- simple trigger-action pairs. "When a form is submitted, send an email." These are useful, but they capture perhaps 5% of the value automation can deliver.

This guide is for practitioners who want to operate at **Levels 3 through 5** -- where automations become intelligent systems that handle edge cases, recover from failures, make decisions, and scale across clients.

LEVEL	DESCRIPTION	EXAMPLE
1 - Reactive	Single trigger, single action	Form submit → email notification
2 - Sequential	Multi-step linear workflows	Form → CRM → email → Slack
3 - Conditional	Branching logic, routers, filters	Lead score > 80 → sales call; else → nurture drip
4 - Intelligent	AI-powered decision making, database state	AI qualifies lead, checks CRM history, routes to best rep
5 - Autonomous	Self-healing agents, multi-scenario orchestration	AI agent handles entire customer lifecycle autonomously

The Three Pillars of Production Automation

Every system you build in this guide rests on three pillars:

1. **Reliability** -- Your automation must handle failures gracefully. If an API is down, the system retries or queues. If input data is malformed, it routes to error handling rather than silently failing. Production automations run 24/7 without you watching.
2. **Observability** -- You must know what every automation is doing. Logging, alerting, execution tracking. When a client asks "did my lead follow-up go out?", you need a definitive answer in seconds.
3. **Scalability** -- A workflow that works for 10 leads/day must work for 10,000. This means understanding operations limits, batch processing, scheduling, and cost management.

How to Use This Guide

Each chapter is self-contained but builds on prior concepts. The code examples use **Make.com** as the primary platform (with notes on n8n equivalents where relevant), **Airtable** for database patterns, and the **OpenAI / Anthropic APIs** for AI integration.

Every JSON payload and prompt template in this guide is **production-tested**. Copy them, modify the variables for your use case, and deploy.

> **WHO THIS GUIDE IS FOR**
>
> Automation consultants, agency owners, and technical implementers who already understand the basics of Make.com (or equivalent). You should be comfortable with JSON, understand HTTP methods, and have built at least a few multi-step workflows. This guide will take you from competent to expert.

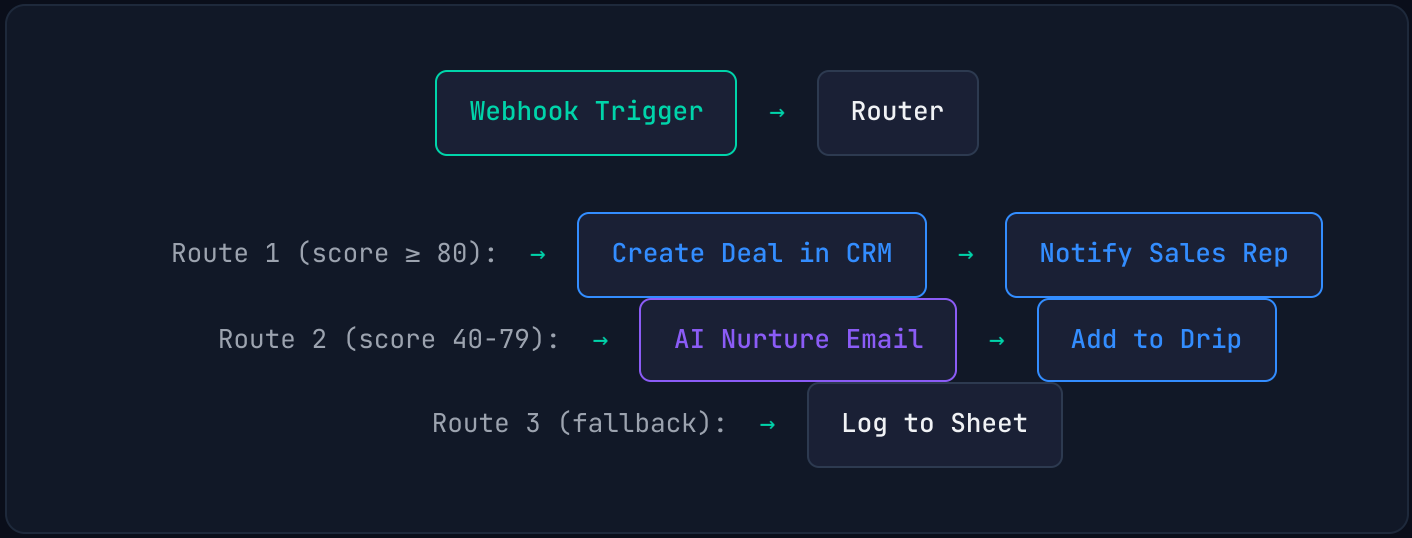
CHAPTER 01

Multi-Step Workflow Architecture

Designing complex workflows with branching logic, error handling, retry mechanisms, and parallel processing patterns.

The Router Pattern

Routers are the backbone of conditional logic in Make.com. A router takes a single input and sends it down multiple paths based on filter conditions. The key insight most people miss: **router paths execute in order, and the first matching path wins by default** -- unless you enable "Run all matching routes."



Router Configuration in Make.com Blueprint

Here is how a router with filters looks in blueprint JSON. Understanding this structure lets you clone and modify scenarios programmatically:

MAKE.COM ROUTER BLUEPRINT FRAGMENT

```
{
  "id": 4,
  "module": "builtin:BasicRouter",
  "version": 1,
  "mapper": null,
  "routes": [
    {
      "flow": [
        {
          "id": 5,
          "module": "hubspot:CreateDeal",
          "version": 3,
          "mapper": {
            "dealname": "{{1.name}} - Inbound Lead",
            "pipeline": "default",
            "dealstage": "qualifiedtobuy",
            "amount": "{{1.estimated_value}}"
          }
        }
      ],
      "filter": {
        "name": "High Score Lead",
        "conditions": [[{
          "a": "{{2.lead_score}}",
          "b": "80",
          "o": "number:greaterThanOrEqual"
        }]]
      }
    },
    {
      "flow": [
        {
          "id": 6,
          "module": "openai:CreateCompletion",
          "version": 1,
          "mapper": {
            "model": "gpt-4o",
            "messages": [
              {
                "role": "system",
```

```
                "content": "Write a personalized nurture email..."
              },
              {
                "role": "user",
                "content": "Lead: {{1.name}}, Industry: {{1.industry}}"
              }
            ]
          }
        }
      ],
      "filter": {
        "name": "Medium Score Lead",
        "conditions": [[
          { "a": "{{2.lead_score}}", "b": "40", "o": "number:greaterThanOrEqual"
},
          { "a": "{{2.lead_score}}", "b": "80", "o": "number:lessThan" }
        ]]
      }
    }
  ]
}
```

The Iterator Pattern

Iterators process arrays one item at a time. They are essential when an API returns a list (e.g., 50 contacts from a CRM query) and you need to perform actions on each item.

When to use iterators:

- Processing CSV rows from a file upload
- Looping through API response arrays
- Sending individualized messages to a list of contacts
- Creating multiple records from a single webhook payload

OPERATIONS WARNING

> Each iteration counts as one operation in Make.com. If you iterate over 200 items and each loop has 3 modules, that is 600 operations per execution. For large datasets, use the **Array Aggregator** to batch records and make a single bulk API call instead.

The Aggregator Pattern

Aggregators collect outputs from iterators (or any sequence of modules) and combine them into a single array or value. This is how you turn many individual results back into one payload.

Common Aggregator Use Cases

PATTERN	USE CASE	AGGREGATOR TYPE
Collect & Send	Gather all processed emails, send as digest	Array Aggregator
Compute Totals	Sum invoice amounts from line items	Numeric Aggregator
Build Report	Combine API responses into single document	Text Aggregator
Batch Insert	Collect records, insert into Airtable in bulk	Array Aggregator

ARRAY AGGREGATOR BLUEPRINT CONFIGURATION

```
{
  "id": 12,
  "module": "builtin:BasicAggregator",
  "version": 1,
  "mapper": {
    "targetStructure": [
      { "name": "email", "type": "text" },
      { "name": "status", "type": "text" },
      { "name": "processed_at", "type": "date" }
    ]
```

```
    },
    "metadata": {
      "source": [8],
      "expect": [
        { "name": "email", "type": "text", "value": "{{8.contact_email}}" },
        { "name": "status", "type": "text", "value": "{{8.result_status}}" },
        { "name": "processed_at", "type": "date", "value": "{{now}}" }
      ]
    }
  }
```

Parallel Processing with Multiple Routes

When you need to perform independent actions simultaneously (e.g., create a CRM contact AND send a Slack notification AND log to a spreadsheet), use a router with **no filters** on each route. All routes execute in parallel.

This is significantly faster than sequential execution and uses the same number of operations. A 5-module parallel scenario completes in the time of the slowest single path, rather than the sum of all paths.

Retry Mechanism Blueprint

Make.com's built-in retry only works at the scenario level. For module-level retries, you need a custom pattern using the **Repeater** module combined with error handling:

```
CUSTOM RETRY PATTERN

Repeater (max 3 iterations)
  → HTTP Make a Request (API call)
  → Router
      Route 1 (success): Break out of loop
      Route 2 (failure):
        → Sleep (exponential: iteration * 2 seconds)
        → Continue to next iteration
```

```
Error Handler on HTTP module:
  → Resume directive (swallow error, let repeater retry)
  → Set variable: last_error = {{error.message}}
```

ARCHITECTURE PRINCIPLE

Design every scenario as if you are building a pipeline in software engineering. Data flows in, gets transformed, gets routed, gets stored. Each module should have a single responsibility. If a module is doing two things, split it into two modules. This makes debugging and modification dramatically easier.

Scenario Composition: The Microservices Approach

Instead of building one massive 30-module scenario, build multiple small scenarios that communicate via webhooks. This gives you:

- **Independent error handling** -- one failure does not cascade
- **Reusable components** -- your "send formatted Slack alert" scenario can be called from anywhere
- **Easier debugging** -- isolate problems to a specific scenario
- **Independent scheduling** -- different throughput needs per component

INTER-SCENARIO COMMUNICATION VIA WEBHOOK

```
// Scenario A: Lead Intake
{
  "url": "https://hook.us1.make.com/your-processing-webhook",
  "method": "POST",
  "headers": {
    "Content-Type": "application/json",
    "X-Internal-Auth": "{{env.INTERNAL_WEBHOOK_SECRET}}"
  },
  "body": {
    "lead_id": "{{3.id}}",
```

```
      "source": "website_form",
      "qualified": true,
      "score": "{{5.calculated_score}}",
      "timestamp": "{{now}}"
    }
  }
```

CHAPTER 02

AI-Powered Automations

Integrating LLMs into production workflows: prompt engineering, structured outputs, chain-of-thought decisions, and real API implementations.

Choosing Your AI Provider

PROVIDER	BEST FOR	COST (PER 1M TOKENS)	MAKE.COM MODULE
OpenAI GPT-4o	General tasks, function calling	~$2.50 in / $10 out	Native module
OpenAI GPT-4o-mini	High volume, simple tasks	~$0.15 in / $0.60 out	Native module
Anthropic Claude	Long context, nuanced writing	~$3 in / $15 out	HTTP module
Google Gemini	Multimodal, large docs	Varies by tier	HTTP module

Structured Outputs: The Key to Reliable AI Automation

The single most important concept for AI in automation: **you must get structured, parseable output**. Free-form text is useless in a pipeline. You need JSON that downstream modules can consume.

OpenAI Structured Output API Call

```
HTTP MODULE CONFIGURATION FOR STRUCTURED OUTPUT

POST https://api.openai.com/v1/chat/completions

Headers:
  Authorization: Bearer {{env.OPENAI_API_KEY}}
  Content-Type: application/json

Body:
{
  "model": "gpt-4o",
  "response_format": {
    "type": "json_schema",
    "json_schema": {
      "name": "lead_qualification",
      "strict": true,
      "schema": {
        "type": "object",
        "properties": {
          "qualified": { "type": "boolean" },
          "score": { "type": "integer", "minimum": 0, "maximum": 100 },
          "reasoning": { "type": "string" },
          "recommended_action": {
            "type": "string",
            "enum": ["schedule_call", "send_nurture", "disqualify", "request_inf
o"]
          },
          "priority": {
            "type": "string",
            "enum": ["high", "medium", "low"]
          }
        },
        "required": ["qualified", "score", "reasoning", "recommended_action", "p
riority"],
        "additionalProperties": false
      }
    }
  },
  "messages": [
    {
      "role": "system",
      "content": "You are a lead qualification specialist for a B2B SaaS compan
```

```
y. Analyze the lead information and determine qualification status. Consider: co
mpany size, budget signals, timeline urgency, decision-maker status, and fit wit
h our ICP (companies with 50-500 employees in tech, healthcare, or finance)."
    },
    {
      "role": "user",
      "content": "Lead Information:\nName: {{1.name}}\nCompany: {{1.company}}\nT
itle: {{1.job_title}}\nEmployees: {{1.company_size}}\nIndustry: {{1.industry}}\n
Message: {{1.message}}\nSource: {{1.utm_source}}"
    }
  ]
}
```

Anthropic Claude API Call

```
CLAUDE API FOR COMPLEX CONTENT GENERATION
POST https://api.anthropic.com/v1/messages

Headers:
  x-api-key: {{env.ANTHROPIC_API_KEY}}
  anthropic-version: 2023-06-01
  Content-Type: application/json

Body:
{
  "model": "claude-sonnet-4-20250514",
  "max_tokens": 2048,
  "system": "You are a professional copywriter. Always respond with valid JSON m
atching the exact schema requested. Never include markdown formatting or code fe
nces in your response.",
  "messages": [
    {
      "role": "user",
      "content": "Write 3 follow-up email variants for this lead. Return JSON:\n
{\"emails\": [{\"subject\": \"...\", \"body\": \"...\", \"tone\": \"formal|casua
l|urgent\", \"cta\": \"...\"}]}\n\nLead context: {{1.name}} from {{1.company}},
interested in {{1.service}}, last contacted {{1.last_contact_date}}."
    }
```

```
  ]
}
```

Chain-of-Thought for Complex Decisions

For decisions that require multi-factor analysis, use chain-of-thought prompting. This dramatically improves accuracy for tasks like proposal pricing, support ticket routing, and lead scoring.

PROMPT TEMPLATE Chain of Thought

```
You are analyzing a customer support ticket to determine priority and routing.

Think through this step by step:

1. URGENCY: Is the customer's business currently impacted? Are they losing revenue?
2. COMPLEXITY: Can a junior rep handle this, or does it need a specialist?
3. SENTIMENT: How frustrated is the customer? Are they at churn risk?
4. HISTORY: Consider their account value and past interaction history.

Ticket: {{ticket.subject}}
Message: {{ticket.body}}
Account Value: {{ticket.account_mrr}}/mo
Previous Tickets (last 30d): {{ticket.recent_count}}
Customer Since: {{ticket.customer_since}}

After your analysis, output ONLY this JSON:
{
  "priority": "P1|P2|P3|P4",
  "department": "billing|technical|account_management|general",
  "estimated_complexity": "simple|moderate|complex",
  "churn_risk": "high|medium|low",
  "suggested_response_template": "string",
```

```
  "reasoning": "string"
}
```

AI for Lead Qualification Pipeline

Here is a complete Make.com scenario design for AI-powered lead qualification:

Content Generation at Scale

For generating content across multiple channels, use a **content multiplication pattern**: generate one core piece of content with AI, then use additional AI calls to adapt it for each channel.

```
CONTENT MULTIPLICATION WORKFLOW

// Step 1: Generate core content from source material
POST /v1/chat/completions
{
  "model": "gpt-4o",
  "messages": [{
    "role": "user",
    "content": "Create a detailed content brief from this blog post: {{source_te
xt}}. Include: key points (5), target audience, primary CTA, tone guidelines, st
atistics to reference."
  }]
}

// Step 2: Fan out to multiple adaptation calls (parallel routes)
// Route A: LinkedIn post (professional, 1300 char max)
// Route B: Email newsletter (conversational, 200 words)
// Route C: Twitter/X thread (5 tweets, hooks + value)
// Route D: Instagram caption (emotional, with hashtags)
// Route E: Blog summary (SEO-optimized, with meta description)
```

COST OPTIMIZATION

Use **GPT-4o-mini** for high-volume, simpler tasks (email classification, sentiment analysis, data extraction). Reserve **GPT-4o or Claude** for tasks requiring nuance (proposal writing, complex qualification, customer-facing content). A typical agency can reduce AI costs by 60-70% with intelligent model routing.

CHAPTER 03

Database-Driven Automations

Using Airtable, Notion, and Make.com data stores as the backbone of stateful automation systems.

Why You Need a Database Layer

Stateless automations (trigger → action) are limited. The moment you need to answer "Has this lead been contacted before?" or "What step of the sequence is this contact on?", you need state. A database layer gives your automations memory.

Airtable as an Automation Database

Airtable excels as an automation database because it provides a UI for non-technical team members while offering a robust API for your automations. Here is a production schema for a lead management system:

Lead Automation Database Schema

FIELD	TYPE	PURPOSE
lead_id	Auto Number	Unique identifier
email	Email	Primary contact, dedup key
full_name	Single Line	Contact name

FIELD	TYPE	PURPOSE
company	Single Line	Company name
source	Single Select	website, referral, cold_outreach, ad
status	Single Select	new, contacted, qualified, nurturing, closed_won, closed_lost
sequence_step	Number	Current step in outreach sequence (1-7)
last_contacted	Date	Last outreach date
next_action_date	Date	When next automation should fire
ai_score	Number	AI-generated qualification score
ai_summary	Long Text	AI-generated lead summary
engagement_data	Long Text (JSON)	Email opens, clicks, replies
assigned_rep	Linked Record	Link to Team Members table
automation_log	Long Text	Append-only log of automation actions

Deduplication Pattern

```
MAKE.COM: CHECK FOR EXISTING LEAD BEFORE CREATING

Module 1: Airtable - Search Records
  Base: Lead Management
  Table: Leads
  Formula: {email} = "{{1.email}}"
  Limit: 1

Module 2: Router
  Route 1 (existing lead found - array length > 0):
```

```
    → Airtable - Update Record (update status, append to log)
  Route 2 (no match - array length = 0):
    → Airtable - Create Record (new lead entry)
```

Make.com Data Stores

Data stores are Make.com's built-in key-value database. They are faster than external API calls and ideal for:

- **Configuration storage** -- API keys, template IDs, client settings
- **Rate limiting counters** -- track API calls per hour
- **Deduplication caches** -- store processed webhook IDs to prevent double-processing
- **State machines** -- track where a multi-step process currently stands

DATA STORE STRUCTURE: WEBHOOK DEDUP CACHE

```
Data Structure:
{
  "key": "string (webhook event ID)",
  "value": {
    "processed_at": "datetime",
    "scenario_execution_id": "string",
    "result_status": "string"
  }
}

// Usage in scenario:
Module 1: Data Store - Search (key = {{webhook.event_id}})
Module 2: Router
  Route 1 (record exists): Stop execution (already processed)
  Route 2 (no record):
    → Process the webhook
    → Data Store - Add/Replace (store the event ID)
```

State Machine Pattern

For complex multi-step processes (like onboarding flows or approval chains), implement a state machine in your database:

```
STATE MACHINE: CLIENT ONBOARDING

States: intake → contract_sent → contract_signed → payment_received
        → setup_in_progress → active

// Each state has:
{
  "client_id": "cli_12345",
  "current_state": "contract_sent",
  "state_history": [
    {"state": "intake", "entered_at": "2026-04-01T10:00:00Z", "trigger": "form_s
ubmission"},
    {"state": "contract_sent", "entered_at": "2026-04-01T10:05:00Z", "trigger":
"automation"}
  ],
  "next_action": "check_contract_status",
  "next_action_date": "2026-04-04T10:00:00Z",
  "timeout_action": "send_reminder"
}

// Scheduled scenario runs every hour:
// 1. Query: WHERE next_action_date ≤ NOW() AND current_state ≠ "active"
// 2. For each result, execute the next_action
// 3. Transition to next state or execute timeout_action
```

LOOKUP TABLES

Create a dedicated "Config" table in Airtable with key-value pairs for values that change per client or environment: API endpoints, email templates, Slack channel IDs,

timezone offsets. Your automations query this table at runtime rather than hardcoding values. This makes white-labeling trivial.

CHAPTER 04

API Integrations Deep Dive

Working with REST APIs, authentication patterns, JSON parsing, pagination, rate limiting, and webhook security.

Authentication Patterns

Every API integration starts with authentication. Here are the four patterns you will encounter:

1. API Key Authentication

HEADER-BASED API KEY

```
GET https://api.example.com/v1/contacts
Headers:
  X-API-Key: {{env.SERVICE_API_KEY}}
  Accept: application/json

// Some APIs use query parameter instead:
GET https://api.example.com/v1/contacts?api_key={{env.SERVICE_API_KEY}}
```

2. Bearer Token (OAuth 2.0 Access Token)

BEARER TOKEN AUTHENTICATION

```
GET https://api.example.com/v1/contacts
Headers:
```

```
    Authorization: Bearer {{connection.access_token}}
    Accept: application/json
```

3. OAuth 2.0 Full Flow

For services like Google, Microsoft, and Salesforce, you need the full OAuth dance. Make.com handles this through its Connection system, but when building custom integrations:

```
OAUTH 2.0 TOKEN REFRESH PATTERN

// Step 1: Check if token is expired
// Store tokens in Data Store with expiry timestamp
Module 1: Data Store - Get Record (key: "oauth_tokens_{{client_id}}")

Module 2: Router
  Route 1 (token expired: expires_at < now):
    Module 3: HTTP POST https://oauth.provider.com/token
      Body (x-www-form-urlencoded):
        grant_type=refresh_token
        refresh_token={{data_store.refresh_token}}
        client_id={{env.OAUTH_CLIENT_ID}}
        client_secret={{env.OAUTH_CLIENT_SECRET}}

    Module 4: Data Store - Update Record
      {
        "access_token": "{{3.access_token}}",
        "refresh_token": "{{3.refresh_token}}",
        "expires_at": "{{addSeconds(now; 3.expires_in)}}"
      }

  Route 2 (token valid):
    Continue with existing access_token
```

4. HMAC Signature Verification (Webhook Security)

VERIFY WEBHOOK SIGNATURE

```
// Many services sign webhook payloads (Stripe, Shopify, GitHub)
// In Make.com, use a custom function or HTTP module to verify:

// Stripe signature verification pattern:
Module 1: Webhook received (raw body available)
Module 2: Set Variable
  expected_sig = HMAC-SHA256(
    timestamp + "." + raw_body,
    {{env.STRIPE_WEBHOOK_SECRET}}
  )
Module 3: Router
  Route 1 (signature matches): Process webhook
  Route 2 (mismatch): Log security alert, stop execution
```

Parsing Complex JSON Responses

Real API responses are nested and complex. Here is how to navigate them in Make.com:

NAVIGATING NESTED JSON IN MAKE.COM

```
// API Response:
{
  "data": {
    "contacts": [
      {
        "id": "c_123",
        "properties": {
          "name": { "first": "John", "last": "Smith" },
          "emails": [
            { "type": "work", "value": "john@company.com", "primary": true }
          ],
          "custom_fields": {
            "lead_score": 85,
            "lifecycle_stage": "opportunity"
          }
        }
      }
```

```
    ],
    "pagination": {
      "next_cursor": "eyJpZCI6MTAwfQ==",
      "has_more": true
    }
  }
}

// Make.com mapping expressions:
First Name:    {{body.data.contacts[].properties.name.first}}
Primary Email: {{body.data.contacts[].properties.emails[1].value}}
Lead Score:    {{body.data.contacts[].properties.custom_fields.lead_score}}
Has More:      {{body.data.pagination.has_more}}
Next Cursor:   {{body.data.pagination.next_cursor}}
```

Handling Pagination

Most APIs return results in pages. Here is the universal pagination pattern:

CURSOR-BASED PAGINATION LOOP

```
// Scenario Structure:
Module 1: Set Variable (cursor = "", all_results = [])

Module 2: Repeater (max iterations: 50)
  Module 3: HTTP GET {{api_url}}?cursor={{cursor}}&limit=100
  Module 4: Array Aggregator (append results to all_results)
  Module 5: Set Variable (cursor = {{3.pagination.next_cursor}})
  Module 6: Router
    Route 1 (has_more = false): Break
    Route 2 (has_more = true): Continue loop

// For offset-based pagination:
GET {{api_url}}?offset={{iteration * 100}}&limit=100
// Stop when response array length < limit
```

Rate Limiting Strategies

STRATEGY	IMPLEMENTATION	BEST FOR
Sleep Between Calls	Add Sleep module (1-2s) after each API call	Simple loops, low volume
Batch + Schedule	Process 100 records per scheduled run, every 5 min	Large datasets
Token Bucket (Data Store)	Track calls in data store, check before each call	Shared rate limits across scenarios
Retry on 429	Check response code, sleep for Retry-After header value	Burst traffic

RATE LIMIT HANDLER PATTERN

```
Module: HTTP Request
Error Handler:
  Filter: Status Code = 429
  Module: Sleep (duration = {{ifempty(error.headers.retry-after; 5)}} seconds)
  Module: HTTP Request (retry the same call)
  Directive: Rollback (retry the module that failed)
```

COMMON API PITFALLS

1. Not handling empty responses. Always check if the response body has the expected structure before accessing nested properties. Use `ifempty()` and `if()` functions.

2. Ignoring HTTP status codes. A 200 response can still contain an error in the body. Always parse and validate.

3. Storing secrets in scenario data. Use environment variables or connection credentials. Never put API keys directly in module configurations.

CHAPTER 05

Advanced Communication Flows

Multi-channel outreach sequences with conditional logic, engagement tracking, A/B testing, and personalization at scale.

Multi-Channel Sequence Architecture

The most effective outreach sequences combine multiple channels in a carefully timed sequence. Each step is conditional on the engagement (or lack thereof) from the previous step.

7-Day Multi-Channel Sequence

Day 0 → Personalized Email #1

↓ *Wait 2 days. Check: opened?*

Day 2 → If opened, no reply: SMS Follow-up

If not opened: Email #2 (new subject line)

Day 4 → Ringless Voicemail Drop

Day 5 → AI-Generated LinkedIn Message

Day 7 → If any engagement: Phone Call Task for Rep

If no engagement: Direct Mail Trigger (Lob API)

Sequence State Management

AIRTABLE RECORD: SEQUENCE TRACKING


```
{
  "lead_id": "lead_4521",
  "sequence_id": "seq_multi_7day",
  "current_step": 3,
  "started_at": "2026-04-01T09:00:00Z",
  "steps_completed": [
    {
      "step": 1,
      "channel": "email",
      "sent_at": "2026-04-01T09:00:00Z",
      "engagement": {"opened": true, "opened_at": "2026-04-01T14:22:00Z", "click
ed": false, "replied": false}
    },
    {
      "step": 2,
      "channel": "sms",
      "sent_at": "2026-04-03T10:00:00Z",
      "engagement": {"delivered": true, "replied": false}
    },
    {
      "step": 3,
      "channel": "voicemail",
      "sent_at": "2026-04-05T11:00:00Z",
      "engagement": {"delivered": true, "callback": false}
    }
  ],
  "next_step": 4,
  "next_step_date": "2026-04-06T09:00:00Z",
  "overall_engaged": true,
  "paused": false,
  "pause_reason": null
```

```
}

// Scheduled scenario (runs every hour):
// 1. Query: next_step_date ≤ NOW() AND paused = false AND status = "active"
// 2. For each lead: look up sequence definition, execute current step
// 3. Update record with new step data
```

A/B Testing Within Automations

Test everything: subject lines, send times, channels, AI prompt variations. Here is a pattern for automated A/B testing:

```
A/B TEST ASSIGNMENT MODULE
// At the start of your sequence, assign a variant:
Module: Set Variable
  variant = {{if(mod(lead_id; 2) = 0; "A"; "B")}}

// For more than 2 variants:
  variant = {{switch(mod(lead_id; 3); 0; "A"; 1; "B"; 2; "C")}}

// Route based on variant:
Router:
  Route A: Subject "Quick question about {{company}}"
  Route B: Subject "{{name}}, saw your post about {{topic}}"

// Log variant assignment for analysis:
Airtable: Update Record
  ab_variant: "{{variant}}"
  ab_test_id: "subjectline_test_042026"
```

Personalization at Scale

True personalization goes beyond `{{first_name}}`. Use AI to generate genuinely personalized content based on real data:

PROMPT TEMPLATE

Personalized Outreach

```
Write a cold outreach email (3-4 sentences max) to {{name}}, {{title}} at
{{company}}.

Context about their company (from LinkedIn/website scrape):
{{company_context}}

Their recent activity/post:
{{recent_activity}}

Our service: AI-powered automation for {{their_industry}} businesses.
Specific pain point to address: {{identified_pain_point}}

Rules:
- Reference their specific situation, not generic benefits
- No "I hope this email finds you well" or similar filler
- End with a specific, low-commitment CTA
- Tone: confident peer, not salesy
- Under 100 words total
```

Voicemail Drop via API

SLYBROADCAST API INTEGRATION

```
POST https://api.slybroadcast.com/v1/campaigns
Headers:
  Authorization: Bearer {{env.SLYBROADCAST_API_KEY}}
Body:
{
  "audio_id": "vm_template_{{variant}}",
  "phone_numbers": ["{{lead.phone}}"],
  "caller_id": "{{env.BUSINESS_PHONE}}",
  "schedule": "immediate",
  "campaign_name": "Sequence {{lead.sequence_id}} Step {{lead.current_step}}"
}
```

Direct Mail via Lob API

LOB POSTCARD API INTEGRATION

```
POST https://api.lob.com/v1/postcards
Headers:
  Authorization: Basic {{base64(env.LOB_API_KEY + ":")}}
Body:
{
  "to": {
    "name": "{{lead.name}}",
    "address_line1": "{{lead.address_line1}}",
    "address_city": "{{lead.city}}",
    "address_state": "{{lead.state}}",
    "address_zip": "{{lead.zip}}"
  },
  "from": {
    "name": "Jedai Flow",
    "address_line1": "...",
    "address_city": "Pearland",
    "address_state": "TX",
    "address_zip": "77584"
  },
  "front": "tmpl_front_{{campaign_id}}",
  "back": "tmpl_back_{{campaign_id}}",
  "merge_variables": {
    "name": "{{lead.first_name}}",
    "company": "{{lead.company}}",
    "custom_line": "{{ai_generated_personalization}}"
  }
}
```

COMPLIANCE REMINDER

SMS: Always get opt-in consent. Include opt-out instructions. Follow TCPA regulations.

Email: Include unsubscribe links. Honor opt-outs within 10 business days (CAN-SPAM). Check GDPR if contacting EU leads.

Voicemail: Verify state-level regulations. Some states restrict ringless voicemail for

commercial purposes.

Build compliance checks into your automation -- query opt-out lists before every send.

CHAPTER 06

Error Handling & Monitoring

Building bulletproof automations with error routes, alerting, dead letter queues, retry strategies, and comprehensive logging.

The Error Handling Hierarchy

Every module in Make.com can have an error handler attached. Error handlers use **directives** to control what happens next:

DIRECTIVE	BEHAVIOR	WHEN TO USE
Resume	Swallow the error, continue with default/empty output	Non-critical modules (logging, notifications)
Rollback	Retry the failed module	Transient failures (API timeouts, rate limits)
Commit	Accept the error, stop execution cleanly	When partial completion is acceptable
Break	Move execution to incomplete queue for manual retry	Critical failures needing human review
Ignore	Same as Resume but does not log the error	Expected errors (404 on optional lookup)

Error Route Pattern

```
COMPLETE ERROR HANDLING SCENARIO DESIGN

Main Flow:
  Webhook → Process Data → API Call → Update CRM → Send Email

Error Route (attached to API Call module):
  → Set Variable: error_context = {
      "module": "API Call",
      "error_message": "{{error.message}}",
      "error_code": "{{error.statusCode}}",
      "input_data": "{{JSON.stringify(3.input)}}",
      "execution_id": "{{executionId}}",
      "timestamp": "{{now}}"
    }
  → Router:
      Route 1 (status 429 - rate limited):
        → Sleep ({{error.headers.retry-after}} seconds)
        → Rollback directive (retry)
      Route 2 (status 401 - auth failed):
        → Slack Alert: "AUTH FAILURE: {{scenario.name}}"
        → Break directive (queue for review)
      Route 3 (status 5xx - server error):
        → Data Store: Add to dead letter queue
        → Slack Alert: "SERVER ERROR: retrying in 5 min"
        → Break directive
      Route 4 (all other errors):
        → Airtable: Log error
        → Email alert to admin
        → Commit directive (stop gracefully)
```

Dead Letter Queue Pattern

When an automation fails and cannot be retried immediately, the failed payload goes to a "dead letter queue" (DLQ) for later processing:

```
DEAD LETTER QUEUE: DATA STORE STRUCTURE

{
```

```
    "key": "dlq_{{executionId}}_{{moduleId}}",
    "value": {
      "scenario_id": 12345,
      "scenario_name": "Lead Processing Pipeline",
      "failed_module": "HTTP - Create HubSpot Contact",
      "error_message": "503 Service Unavailable",
      "error_code": 503,
      "input_payload": { /* full input to the failed module */ },
      "retry_count": 0,
      "max_retries": 3,
      "created_at": "2026-04-08T14:30:00Z",
      "next_retry_at": "2026-04-08T14:45:00Z",
      "status": "pending"
    }
  }

  // Separate "DLQ Processor" scenario (scheduled every 15 min):
  // 1. Query data store: status = "pending" AND next_retry_at ≤ NOW()
  // 2. Iterate over results
  // 3. Attempt to reprocess each payload
  // 4. On success: update status to "resolved", delete from DLQ
  // 5. On failure: increment retry_count, set next_retry_at (exponential backoff)
  // 6. If retry_count ≥ max_retries: update status to "failed", alert human
```

Slack Alerting System

```
STRUCTURED SLACK ALERT MESSAGE
POST https://slack.com/api/chat.postMessage
Headers:
  Authorization: Bearer {{env.SLACK_BOT_TOKEN}}
Body:
{
  "channel": "#automation-alerts",
  "blocks": [
    {
      "type": "header",
      "text": {"type": "plain_text", "text": "Automation Alert: {{severity}}"}
    },
```

```
      {
        "type": "section",
        "fields": [
          {"type": "mrkdwn", "text": "*Scenario:*\n{{scenario.name}}"},
          {"type": "mrkdwn", "text": "*Module:*\n{{failed_module}}"},
          {"type": "mrkdwn", "text": "*Error:*\n{{error.message}}"},
          {"type": "mrkdwn", "text": "*Time:*\n{{formatDate(now; \"MMM D, h:mm A
\")}}"}
        ]
      },
      {
        "type": "section",
        "text": {
          "type": "mrkdwn",
          "text": "*Execution ID:* `{{executionId}}`\n"
        }
      },
      {
        "type": "actions",
        "elements": [
          {
            "type": "button",
            "text": {"type": "plain_text", "text": "Retry Now"},
            "url": "{{retry_webhook_url}}?execution_id={{executionId}}"
          }
        ]
      }
    ]
  }
```

Execution Logging Pattern

Log every automation run to a dedicated table. This is essential for client reporting and debugging:

```
AIRTABLE: AUTOMATION EXECUTION LOG

// First module in every scenario (after trigger):
Airtable - Create Record (Execution Log table):
```

```
{
  "execution_id": "{{executionId}}",
  "scenario_name": "Lead Processing Pipeline",
  "trigger_type": "webhook",
  "input_summary": "Lead: {{1.email}} from {{1.source}}",
  "started_at": "{{now}}",
  "status": "running"
}

// Last module in every scenario (before end):
Airtable - Update Record:
{
  "status": "completed",
  "completed_at": "{{now}}",
  "output_summary": "Created contact, sent welcome email",
  "operations_used": "{{scenario.operations}}"
}

// In error handler:
Airtable - Update Record:
{
  "status": "failed",
  "completed_at": "{{now}}",
  "error_details": "{{error.message}} at module {{error.moduleName}}"
}
```

MONITORING DASHBOARD

Build a simple Airtable Interface or Google Looker Studio dashboard from your execution log. Track: success rate per scenario, average execution time, operations consumed per day, error frequency by type. Review this weekly to catch degrading performance before it becomes a crisis.

CHAPTER 07

White-Label & Client Management

Running automations for multiple clients: organization structure, workspace separation, reporting dashboards, and SLA management.

Organization Architecture

When managing automations for multiple clients, structure is everything. There are two approaches:

Approach 1: Separate Workspaces (Recommended)

Each client gets their own Make.com team/workspace. This provides complete isolation -- a misconfigured scenario for Client A cannot affect Client B.

- Each workspace has its own connections, data stores, and scenarios
- Client can be given viewer access to their workspace
- Billing can be tracked per-workspace
- Migrating a client to self-management is straightforward

Approach 2: Shared Workspace with Client Tags

All clients in one workspace, with scenarios and folders organized by client. Use a configuration data store to route data per client.

```
MULTI-TENANT CONFIGURATION DATA STORE
{
  "key": "client_acme_corp",
```

```
    "value": {
      "client_id": "acme",
      "client_name": "Acme Corporation",
      "crm_api_key": "{{encrypted}}",
      "crm_type": "hubspot",
      "email_provider": "mailgun",
      "email_domain": "mail.acmecorp.com",
      "slack_webhook": "https://hooks.slack.com/...",
      "branding": {
        "company_name": "Acme Corporation",
        "primary_color": "#ff6b00",
        "logo_url": "https://..."
      },
      "settings": {
        "timezone": "America/Chicago",
        "working_hours": {"start": "08:00", "end": "18:00"},
        "max_emails_per_day": 200,
        "ai_model": "gpt-4o-mini",
        "lead_score_threshold": 70
      }
    }
  }

  // At the start of every scenario:
  // 1. Identify the client (from webhook data, trigger source, etc.)
  // 2. Load client config from data store
  // 3. Use config values throughout the scenario
```

Client Reporting Dashboard

Every client should receive a regular report. Automate the creation of these reports:

```
  WEEKLY CLIENT REPORT AUTOMATION
  // Scheduled: Every Monday at 8 AM

  Module 1: Data Store - List (all active clients)
  Module 2: Iterator (for each client)
    Module 3: Airtable - Search Records
```

```
    Formula: AND(
      {client_id} = "{{client.id}}",
      {created_at} ≥ "{{addDays(now; -7)}}",
      {type} = "execution_log"
    )
  Module 4: Aggregator (compute stats)
    - total_executions
    - successful_executions
    - failed_executions
    - total_operations_used
    - unique_scenarios_run
  Module 5: OpenAI - Generate Report Summary
    "Summarize these automation metrics for a non-technical client..."
  Module 6: Email - Send Report
    To: {{client.report_email}}
    Subject: "Weekly Automation Report - {{formatDate(now; 'MMM D')}}"
    Body: HTML template with metrics + AI summary
```

SLA Management

SLA METRIC	TARGET	HOW TO TRACK
Uptime	99.5%	Execution log: (successful / total) * 100
Response Time	< 5 min for triggers	Timestamp diff: trigger received vs. first action
Error Resolution	< 4 hours	Time from error alert to status = "resolved" in DLQ
Monthly Operations	Within plan limits	Sum operations from execution log

CLIENT ONBOARDING TEMPLATE

Create a standard onboarding automation that: (1) Creates the client workspace/folder, (2) Deploys your base scenario templates via the Make.com API, (3) Sets up the configuration data store with client info, (4) Creates the client's

Airtable base from a template, (5) Sends the welcome email with dashboard access. This turns a 2-hour manual process into a 5-minute automation.

CHAPTER 08

Scaling Infrastructure

Operations optimization, scheduling strategies, microservice decomposition, and cost management for growing automation systems.

Understanding Operations Economics

Make.com charges by operations. Every module execution is one operation. Understanding this is critical to profitability.

PLAN	OPERATIONS/MO	PRICE	COST PER 1K OPS
Free	1,000	$0	$0
Core	10,000	$10.59/mo	$1.06
Pro	10,000	$18.82/mo	$1.88
Teams	10,000	$34.12/mo	$3.41
Enterprise	Custom	Custom	Negotiable

Additional operations: approximately $3.50 per 1,000 on Pro plans. Prices as of early 2026.

Operations Optimization Techniques

1. Filter Early, Process Less

Place filters as close to the trigger as possible. If only 20% of webhook events need processing, a filter after module 1 saves you from running modules 2-10 on the other 80%.

2. Batch Instead of Iterate

```
BEFORE: ITERATE PATTERN (500 OPERATIONS)

Trigger (1 op) → Get 100 Records (1 op) → Iterator (100 items)
  For each: Update CRM (1 op) + Send Email (1 op) + Log (1 op)
  = 100 x 3 = 300 ops + overhead
Total: ~400-500 operations

AFTER: BATCH PATTERN (5 OPERATIONS)

Trigger (1 op) → Get 100 Records (1 op) → Array Aggregator (1 op)
  → HTTP Bulk API Call (1 op) → Log Summary (1 op)
Total: 5 operations
```

3. Consolidate Data Lookups

Instead of making 5 separate API calls to gather data, use tools that support batch queries or build a composite lookup scenario.

4. Use Scheduled Polling Instead of Instant Triggers

Instant triggers (webhooks) fire individually per event. A scheduled scenario that polls every 5 minutes and processes all new records in batch uses fewer operations for high-volume sources.

Breaking Monoliths into Microservices

```
BEFORE: MONOLITH SCENARIO (30+ MODULES)

Webhook → Validate → Enrich → Score → Route → CRM → Email
→ SMS → Slack → Log → Report → ... (30 modules)
```

```
Problems:
- Single point of failure
- Cannot update one part without risking the whole
- Operations limit hit on one scenario
- Debugging is a nightmare

AFTER: MICROSERVICE SCENARIOS

Scenario 1: Lead Intake (Webhook → Validate → Queue to Scenario 2)
Scenario 2: Lead Enrichment (Process → Enrich via APIs → Queue to S3)
Scenario 3: Lead Scoring (AI Score → Update DB → Queue to S4)
Scenario 4: Lead Routing (Check score → Assign rep → Trigger S5/S6)
Scenario 5: Communications (Send email/SMS based on queue)
Scenario 6: Logging & Reporting (Centralized execution log)

Benefits:
- Each scenario is simple and testable
- Independent error handling and retry
- Can scale scenarios independently
- Easy to modify one without touching others
```

Scheduling Strategies

PATTERN	SCHEDULE	USE CASE
Real-time	Webhook (instant)	Form submissions, payment events
Near real-time	Every 1-5 minutes	CRM sync, inbox monitoring
Batch processing	Every 15-60 minutes	Bulk data processing, report generation
Daily digest	Once per day	Summary reports, cleanup tasks
Business hours only	Every 5 min, 8 AM-6 PM weekdays	Outreach sequences (respect recipients)

COST MANAGEMENT FORMULA

For client pricing, use this formula: **(estimated monthly operations / 1000) x $3.50 × 2.5 markup = monthly automation fee**. The 2.5x markup covers your time for maintenance, monitoring, and optimization. A client using 50K operations/month costs you ~$175 in Make.com fees; charge $437/month minimum. Layer AI API costs on top (typically $20-100/month per client).

CHAPTER 09

Building AI Agents

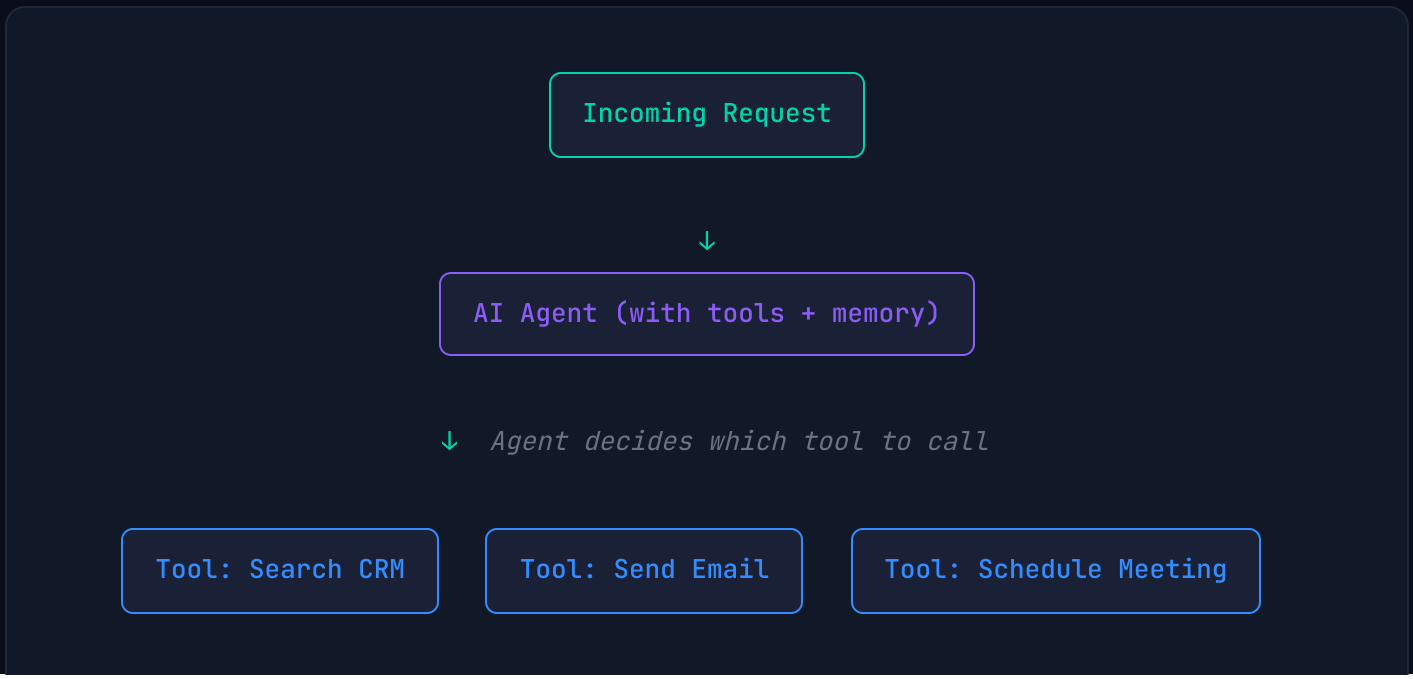
Creating autonomous AI agents that handle entire workflows using tool-calling, function definitions, memory, and multi-turn reasoning.

What Makes an Agent Different from a Prompt

A prompt is a single AI call. An agent is an **AI that can take actions**. The difference is tool-calling: you give the AI a set of functions it can invoke, and it decides which ones to use based on the situation.

This is the most powerful pattern in modern automation. Instead of writing complex branching logic yourself, you describe the tools and let the AI orchestrate.

Agent Architecture



Tool: Check Calendar

↓ *Tool result returned to agent*

Agent processes result, may call more tools

↓

Final Response / Action Taken

Customer Support Agent: Complete Implementation

OPENAI FUNCTION CALLING: SUPPORT AGENT

```
POST https://api.openai.com/v1/chat/completions
{
  "model": "gpt-4o",
  "messages": [
    {
      "role": "system",
      "content": "You are a customer support agent for {{company_name}}. You have access to tools to look up customer information, check order status, process refunds, and escalate to human agents. Always be helpful and professional. If you cannot resolve an issue, escalate. Never make promises about timelines you cannot verify with tools."
    },
    {
      "role": "user",
      "content": "{{customer_message}}"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "lookup_customer",
```

```
          "description": "Look up customer information by email or customer ID",
          "parameters": {
            "type": "object",
            "properties": {
              "email": { "type": "string", "description": "Customer email address"
  },
              "customer_id": { "type": "string", "description": "Customer ID" }
            }
          }
        }
      },
      {
        "type": "function",
        "function": {
          "name": "check_order_status",
          "description": "Check the status of a specific order",
          "parameters": {
            "type": "object",
            "properties": {
              "order_id": { "type": "string" }
            },
            "required": ["order_id"]
          }
        }
      },
      {
        "type": "function",
        "function": {
          "name": "process_refund",
          "description": "Initiate a refund for an order. Only use when customer e
  xplicitly requests refund and order is eligible.",
          "parameters": {
            "type": "object",
            "properties": {
              "order_id": { "type": "string" },
              "amount": { "type": "number", "description": "Refund amount in dolla
  rs" },
              "reason": { "type": "string" }
            },
            "required": ["order_id", "amount", "reason"]
          }
        }
```

```
    },
    {
      "type": "function",
      "function": {
        "name": "escalate_to_human",
        "description": "Transfer the conversation to a human agent",
        "parameters": {
          "type": "object",
          "properties": {
            "department": { "type": "string", "enum": ["billing", "technical",
"management"] },
            "priority": { "type": "string", "enum": ["normal", "urgent"] },
            "summary": { "type": "string", "description": "Brief summary for the
human agent" }
          },
          "required": ["department", "priority", "summary"]
        }
      }
    },
    {
      "type": "function",
      "function": {
        "name": "send_email",
        "description": "Send a follow-up email to the customer",
        "parameters": {
          "type": "object",
          "properties": {
            "to": { "type": "string" },
            "subject": { "type": "string" },
            "body": { "type": "string" }
          },
          "required": ["to", "subject", "body"]
        }
      }
    }
  ],
  "tool_choice": "auto"
}
```

Handling Tool Calls in Make.com

AGENT LOOP PATTERN IN MAKE.COM

```
Module 1: Receive customer message (webhook/email/chat)
Module 2: Load conversation history from data store
Module 3: OpenAI - Create Chat Completion (with tools)
Module 4: Router
  Route 1 (response has tool_calls):
    → Iterator over tool_calls array
    → Router (by function name):
        "lookup_customer": → Airtable Search → Return result
        "check_order_status": → Shopify API → Return result
        "process_refund": → Stripe Refund API → Return result
        "escalate_to_human": → Create Zendesk ticket → Return result
        "send_email": → Mailgun API → Return result
    → Aggregator: Collect all tool results
    → OpenAI - Create Chat Completion (with tool results appended)
    → Loop back to Module 4 (agent may call more tools)

  Route 2 (response is final text, no tool_calls):
    → Send response to customer
    → Save conversation to data store
```

Sales Qualification Agent

AGENT SYSTEM PROMPT

Sales Qualification

```
You are a sales qualification agent for {{company_name}}. Your job is to
engage with inbound leads, gather qualifying information, and determine next
steps.

QUALIFICATION CRITERIA (BANT):
- Budget: Can they afford our services? ($2K-10K/month range)
- Authority: Are they a decision-maker or influencer?
- Need: Do they have a clear pain point we solve?
- Timeline: Are they looking to implement within 90 days?

YOUR TOOLS:
```

```
- search_crm: Check if this lead exists in our CRM
- enrich_company: Get company info (size, industry, funding)
- check_calendar: See available slots for discovery calls
- book_meeting: Schedule a discovery call
- add_to_nurture: Add to email nurture sequence
- update_crm: Update lead record with qualification data

RULES:
1. Be conversational and helpful, not interrogative
2. Gather info naturally across 2-3 message exchanges
3. If qualified (3+ BANT criteria met): book a meeting
4. If partially qualified: add to nurture with specific notes
5. If unqualified: politely redirect to self-serve resources
6. Always update CRM with everything you learn
```

Agent Memory: Conversation Persistence

CONVERSATION MEMORY DATA STORE STRUCTURE

```
{
  "key": "conv_{{customer_id}}",
  "value": {
    "customer_id": "cust_789",
    "messages": [
      {"role": "user", "content": "I need help with my order", "timestamp":
"..."},
      {"role": "assistant", "content": "I'd be happy to help...", "timestamp":
"..."},
      {"role": "user", "content": "Order #12345 hasn't arrived", "timestamp":
"..."}
    ],
    "context": {
      "customer_name": "Sarah Johnson",
      "account_tier": "premium",
      "open_issues": ["order_12345_delayed"],
      "sentiment_trend": "frustrated",
      "total_interactions": 3
    },
```

```
    "last_updated": "2026-04-08T10:30:00Z",
    "ttl": "2026-04-15T10:30:00Z"
  }
}

// Memory management:
// - Load last 10 messages as context (token budget)
// - Summarize older messages with AI: "Previous context: Customer has been
//   waiting 5 days for order #12345. Previously offered tracking update."
// - Set TTL to auto-expire stale conversations
// - Store key extracted facts in "context" for quick loading
```

Scheduling Agent Example

```
TOOL DEFINITIONS FOR SCHEDULING AGENT
{
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_available_slots",
        "description": "Get available meeting slots for a specific date range an
d team member",
        "parameters": {
          "type": "object",
          "properties": {
            "team_member": { "type": "string", "description": "Name or email of
team member" },
            "start_date": { "type": "string", "format": "date" },
            "end_date": { "type": "string", "format": "date" },
            "duration_minutes": { "type": "integer", "enum": [15, 30, 60] },
            "timezone": { "type": "string", "description": "IANA timezone" }
          },
          "required": ["team_member", "start_date", "end_date", "duration_minute
s"]
        }
      }
    },
```

```
    {
      "type": "function",
      "function": {
        "name": "book_meeting",
        "description": "Book a meeting on the calendar",
        "parameters": {
          "type": "object",
          "properties": {
            "title": { "type": "string" },
            "start_time": { "type": "string", "format": "date-time" },
            "duration_minutes": { "type": "integer" },
            "attendees": { "type": "array", "items": { "type": "string" } },
            "description": { "type": "string" },
            "video_link": { "type": "boolean", "description": "Auto-generate Zoo
m link" }
          },
          "required": ["title", "start_time", "duration_minutes", "attendees"]
        }
      }
    },
    {
      "type": "function",
      "function": {
        "name": "reschedule_meeting",
        "description": "Reschedule an existing meeting",
        "parameters": {
          "type": "object",
          "properties": {
            "meeting_id": { "type": "string" },
            "new_start_time": { "type": "string", "format": "date-time" },
            "notify_attendees": { "type": "boolean" },
            "reason": { "type": "string" }
          },
          "required": ["meeting_id", "new_start_time"]
        }
      }
    }
  ]
}
```

AGENT SAFETY: GUARD RAILS

Always implement guard rails for autonomous agents:

1. **Spending limits:** Cap refund amounts, require human approval above thresholds
2. **Action logging:** Every tool call must be logged with full input/output
3. **Confirmation for destructive actions:** Deletions, refunds, cancellations should require confirmation
4. **Human escalation paths:** The agent must always be able to hand off to a human
5. **Rate limits:** Cap the number of tool calls per conversation (prevent infinite loops)
6. **Content filtering:** Check agent responses before sending to customers

BONUS CHAPTER

Advanced Prompt Library

20 production-ready prompts for automation workflows. Copy, customize the variables, and deploy.

Each prompt below is designed for use in Make.com's OpenAI/HTTP modules. Variables in `{{brackets}}` should be mapped to data from previous modules in your scenario.

PROMPT 01: LEAD QUALIFICATION

Sales

```
Analyze this lead and return a JSON qualification assessment.

Lead Data:
- Name: {{name}}
- Company: {{company}} (Industry: {{industry}}, Size: {{employees}} employees)
- Title: {{job_title}}
- Message: {{form_message}}
- Source: {{utm_source}} / {{utm_campaign}}

Our ICP: {{ideal_customer_profile}}
Our Services: {{services_offered}}

Return ONLY valid JSON:
{
  "score": 0-100,
  "qualified": true/false,
  "icp_fit": "strong|moderate|weak|none",
  "decision_maker": true/false,
  "budget_signal": "high|medium|low|unknown",
  "urgency": "immediate|short_term|exploring|unknown",
```

```
  "recommended_action":
"book_call|send_case_study|nurture_sequence|disqualify",
  "personalized_subject_line": "string for follow-up email",
  "key_talking_points": ["point 1", "point 2", "point 3"],
  "reasoning": "2-3 sentence explanation"
}
```

PROMPT 02: EMAIL REWRITING Communications

```
Rewrite this email to be more professional, concise, and action-oriented.

Original email:
{{original_email}}

Context: This is a {{email_type}} email to {{recipient_role}}.
Desired tone: {{tone}}
Maximum length: {{max_words}} words
Must include CTA: {{call_to_action}}

Return JSON:
{
  "subject": "improved subject line",
  "body": "rewritten email body (plain text)",
  "body_html": "rewritten email body (HTML formatted)",
  "changes_made": ["list of key changes"],
  "readability_score": "easy|moderate|advanced"
}
```

PROMPT 03: REVIEW RESPONSE GENERATOR Reputation

```
Generate a response to this customer review.

Platform: {{platform}} (Google/Yelp/Facebook)
Rating: {{star_rating}}/5 stars
Review text: {{review_text}}
```

```
Customer name: {{customer_name}}
Business name: {{business_name}}

Guidelines:
- For 4-5 stars: Thank them specifically, reinforce what they loved, invite them back
- For 3 stars: Acknowledge feedback, address specific concerns, offer to improve
- For 1-2 stars: Empathize, apologize without being defensive, take conversation offline
- Always be genuine, never generic
- Include the customer's name
- Keep under 100 words
- Never offer discounts/freebies publicly (invites exploitation)

Return JSON:
{
  "response": "the review response text",
  "sentiment": "positive|neutral|negative",
  "issues_identified": ["list of issues if any"],
  "follow_up_needed": true/false,
  "internal_action": "none|contact_customer|investigate|urgent_flag"
}
```

PROMPT 04: MEETING SUMMARY Productivity

```
Analyze this meeting transcript and produce a structured summary.

Transcript:
{{transcript_text}}

Meeting type: {{meeting_type}}
Attendees: {{attendees_list}}

Return JSON:
{
  "title": "descriptive meeting title",
```

```
  "duration_discussed": "estimated duration from content",
  "summary": "3-5 sentence executive summary",
  "key_decisions": [
    {"decision": "what was decided", "owner": "who owns it", "deadline": "if
mentioned"}
  ],
  "action_items": [
    {"task": "specific task", "assignee": "person responsible", "due_date":
"if mentioned", "priority": "high|medium|low"}
  ],
  "open_questions": ["unresolved items that need follow-up"],
  "next_meeting": "date/topic if discussed",
  "sentiment": "productive|neutral|contentious",
  "follow_up_email_draft": "brief email summarizing outcomes for attendees"
}
```

PROMPT 05: PROPOSAL DRAFTING Sales

```
Draft a project proposal based on the discovery call notes.

Client: {{client_name}} ({{client_company}})
Industry: {{industry}}
Discovery notes: {{discovery_notes}}
Pain points identified: {{pain_points}}
Budget range discussed: {{budget_range}}
Timeline: {{desired_timeline}}
Our relevant services: {{applicable_services}}

Return JSON:
{
  "proposal_title": "string",
  "executive_summary": "2-3 paragraphs addressing their specific situation",
  "proposed_solution": {
    "phases": [
      {
        "name": "Phase name",
        "duration": "X weeks",
```

```
        "deliverables": ["list"],
        "description": "what we do and why"
      }
    ]
  },
  "investment": {
    "setup_fee": "amount or range",
    "monthly_fee": "amount or range",
    "justification": "ROI-focused explanation"
  },
  "timeline": "overall project timeline",
  "next_steps": ["ordered list of immediate next steps"],
  "why_us": "2-3 sentences on unique value proposition"
}
```

PROMPT 06: DATA EXTRACTION FROM UNSTRUCTURED TEXT Data Processing

```
Extract structured data from this unstructured text.

Source text:
{{raw_text}}

Source type: {{source_type}} (email/form/chat/document)

Extract and return JSON:
{
  "people": [{"name": "...", "role": "...", "email": "...", "phone": "..."}],
  "companies": [{"name": "...", "industry": "...", "size": "..."}],
  "dates": [{"date": "YYYY-MM-DD", "context": "what the date refers to"}],
  "monetary_values": [{"amount": 0, "currency": "USD", "context": "..."}],
  "addresses": [{"full": "...", "city": "...", "state": "...", "zip": "..."}],
  "action_items": ["extracted tasks or requests"],
  "key_topics": ["main subjects discussed"],
  "sentiment": "positive|neutral|negative",
  "urgency": "high|medium|low",
  "language": "detected language"
}
```

```
Rules: If a field cannot be determined, use null. Never fabricate data.
```

PROMPT 07: SENTIMENT ANALYSIS WITH CONTEXT

Analytics

```
Perform deep sentiment analysis on this customer communication.

Message: {{message_text}}
Channel: {{channel}}
Customer tier: {{customer_tier}}
Previous interactions (last 30 days): {{interaction_count}}

Return JSON:
{
  "overall_sentiment":
"very_positive|positive|neutral|negative|very_negative",
  "confidence": 0.0-1.0,
  "emotions_detected": ["frustration", "satisfaction", "confusion", "urgency",
"anger", "gratitude"],
  "churn_risk": "high|medium|low",
  "churn_signals": ["specific phrases or patterns indicating churn risk"],
  "topic_sentiments": [
    {"topic": "product quality", "sentiment": "positive", "evidence": "quote
from text"}
  ],
  "recommended_response_tone":
"empathetic|professional|enthusiastic|apologetic|urgent",
  "escalation_needed": true/false,
  "escalation_reason": "string or null"
}
```

PROMPT 08: CONTENT REPURPOSING

Marketing

```
Repurpose this content for multiple platforms.
```

```
Original content:
{{source_content}}

Content type: {{content_type}}
Brand voice: {{brand_voice_description}}
Target audience: {{target_audience}}

Return JSON:
{
  "linkedin_post": {
    "text": "1300 chars max, professional tone, hook + value + CTA",
    "hashtags": ["relevant", "hashtags"]
  },
  "twitter_thread": [
    "Tweet 1: Hook (280 chars)",
    "Tweet 2: Key insight",
    "Tweet 3: Supporting point",
    "Tweet 4: CTA"
  ],
  "email_newsletter": {
    "subject": "compelling subject line",
    "preview_text": "80 chars",
    "body": "200-300 words, conversational"
  },
  "instagram_caption": {
    "text": "engaging caption with line breaks",
    "hashtags": ["up to 15 relevant hashtags"]
  },
  "blog_meta": {
    "seo_title": "60 chars max",
    "meta_description": "155 chars max",
    "slug": "url-friendly-slug"
  }
}
```

PROMPT 09: SUPPORT TICKET CLASSIFICATION Support

```
Classify this support ticket for routing and prioritization.

Subject: {{ticket_subject}}
Body: {{ticket_body}}
Customer email: {{customer_email}}
Account MRR: ${{account_mrr}}
Open tickets: {{open_ticket_count}}

Return JSON:
{
  "category": "billing|technical|feature_request|bug_report|account|general",
  "subcategory": "more specific classification",
  "priority": "P1_critical|P2_high|P3_medium|P4_low",
  "department": "engineering|billing|success|sales",
  "estimated_complexity": "simple|moderate|complex",
  "auto_response_appropriate": true/false,
  "suggested_auto_response": "response text if appropriate, null otherwise",
  "tags": ["relevant", "tags", "for", "tracking"],
  "similar_known_issue": "description of known issue if match found, null
otherwise"
}
```

PROMPT 10: INVOICE DATA EXTRACTION Finance

```
Extract invoice data from this text/OCR output.

Raw text:
{{invoice_text}}

Return JSON:
{
  "vendor_name": "string",
  "vendor_address": "string",
  "invoice_number": "string",
  "invoice_date": "YYYY-MM-DD",
  "due_date": "YYYY-MM-DD",
```

```
  "line_items": [
    {"description": "...", "quantity": 0, "unit_price": 0.00, "total": 0.00}
  ],
  "subtotal": 0.00,
  "tax_amount": 0.00,
  "tax_rate": "percentage or null",
  "total_amount": 0.00,
  "currency": "USD",
  "payment_terms": "Net 30, etc.",
  "po_number": "string or null",
  "confidence": "high|medium|low"
}
```

PROMPT 11: COMPETITIVE INTEL SUMMARIZER

Strategy

```
Analyze this competitive intelligence data and produce an actionable summary.

Competitor: {{competitor_name}}
Data source: {{source_type}}
Raw data:
{{scraped_content}}

Our positioning: {{our_positioning}}

Return JSON:
{
  "competitor_summary": "2-3 sentence overview of competitor's current state",
  "recent_changes": ["new features, pricing changes, hires, etc."],
  "strengths_vs_us": ["where they are stronger"],
  "weaknesses_vs_us": ["where we have advantage"],
  "pricing_intel": {"model": "...", "range": "...", "changes": "..."},
  "threat_level": "high|medium|low",
  "recommended_actions": ["specific things we should do in response"],
  "talking_points": ["points for sales team when asked about this competitor"]
}
```

PROMPT 12: BLOG POST TO FAQ GENERATOR

Content

```
Generate FAQ entries from this blog post / knowledge base article.

Content:
{{article_content}}

Target audience: {{audience}}

Return JSON:
{
  "faqs": [
    {
      "question": "natural question a customer would ask",
      "answer": "concise, helpful answer (2-3 sentences)",
      "category": "topic category",
      "schema_markup": "FAQ schema JSON-LD for this Q&A pair"
    }
  ],
  "suggested_related_topics": ["topics not covered that customers might ask
about"]
}
```

PROMPT 13: COLD EMAIL PERSONALIZATION

Outreach

```
Write a personalized cold email first line and P.S. based on research.

Recipient: {{name}}, {{title}} at {{company}}
LinkedIn headline: {{linkedin_headline}}
Recent post/activity: {{recent_activity}}
Company news: {{company_news}}
Mutual connections: {{mutual_connections}}

Return JSON:
{
  "opening_line": "personalized, reference-specific first line (1 sentence)",
```

```
  "ps_line": "personal touch P.S. line",
  "connection_angle": "why they should care (1 sentence)",
  "personalization_type":
"achievement|shared_interest|company_news|mutual_connection|content_reference"
}
```

PROMPT 14: CHATBOT CONVERSATION ROUTER

Support

```
Analyze this chatbot conversation and determine the best next action.

Conversation so far:
{{conversation_history}}

Latest message: {{latest_message}}

Available actions:
1. continue_chat - Bot can handle this
2. transfer_sales - Lead showing buying intent
3. transfer_support - Technical issue detected
4. collect_email - Capture lead before they leave
5. show_pricing - Pricing question detected
6. book_demo - Demo request detected
7. send_resource - Can be answered with a specific resource

Return JSON:
{
  "action": "one of the above",
  "confidence": 0.0-1.0,
  "intent": "what the user is trying to accomplish",
  "bot_response": "what the bot should say next",
  "internal_notes": "context for human if transferred",
  "lead_score_adjustment": -10 to +10
}
```

PROMPT 15: SOCIAL MEDIA COMMENT MODERATOR

Social

```
Analyze this social media comment and determine the appropriate action.

Comment: {{comment_text}}
Author: {{author_name}}
Platform: {{platform}}
Post context: {{original_post_topic}}

Return JSON:
{
  "action": "approve|reply|hide|delete|escalate",
  "category": "positive|question|complaint|spam|inappropriate|troll",
  "reply_text": "suggested reply if action is reply, null otherwise",
  "urgency": "high|medium|low",
  "public_sentiment_impact": "positive|neutral|negative",
  "reasoning": "why this action was chosen"
}
```

PROMPT 16: RESUME/APPLICATION SCREENER HR

```
Screen this job application against the job requirements.

Job title: {{job_title}}
Requirements: {{job_requirements}}
Nice-to-haves: {{nice_to_haves}}

Application data:
{{resume_text}}

Return JSON:
{
  "overall_match": "strong|good|partial|weak",
  "score": 0-100,
  "requirements_met": [{"requirement": "...", "met": true/false, "evidence":
"..."}],
  "nice_to_haves_met": [{"item": "...", "met": true/false}],
  "strengths": ["notable strengths relevant to role"],
```

```
  "concerns": ["potential issues or gaps"],
  "experience_years": "estimated relevant experience",
  "recommended_action": "advance_to_interview|phone_screen|hold|reject",
  "interview_questions": ["3 role-specific questions based on their
background"]
}
```

PROMPT 17: PRODUCT DESCRIPTION GENERATOR

E-Commerce

```
Generate an optimized product description from raw product data.

Product name: {{product_name}}
Category: {{category}}
Raw specs: {{raw_specifications}}
Target customer: {{target_customer}}
Key selling points: {{selling_points}}
Competitor price range: {{competitor_prices}}

Return JSON:
{
  "title": "SEO-optimized product title (60 chars)",
  "short_description": "50-word benefit-focused summary",
  "long_description": "200-word detailed description with HTML formatting",
  "bullet_points": ["5-7 key feature/benefit bullets"],
  "seo_keywords": ["primary and secondary keywords"],
  "meta_description": "155 chars for search results",
  "suggested_categories": ["where to list this product"],
  "upsell_angles": ["cross-sell and upsell suggestions"]
}
```

PROMPT 18: CONTRACT CLAUSE ANALYZER

Legal

```
Analyze this contract excerpt and flag important clauses.

Contract text:
```

```
{{contract_text}}

Contract type: {{contract_type}}
Our role: {{our_role}} (vendor/client/partner)

IMPORTANT: This is for informational flagging only, not legal advice.

Return JSON:
{
  "clauses_identified": [
    {
      "type":
"termination|liability|payment|ip|non_compete|auto_renewal|indemnity|confidential
      "text": "relevant excerpt",
      "risk_level": "high|medium|low",
      "plain_english": "what this means in simple terms",
      "flag_reason": "why this needs attention"
    }
  ],
  "missing_clauses": ["important clauses not found in the contract"],
  "overall_risk": "high|medium|low",
  "recommended_review": true/false,
  "summary": "2-3 sentence plain-English contract summary"
}
```

PROMPT 19: WEEKLY KPI REPORT NARRATOR Reporting

```
Generate an executive narrative from these KPI metrics.

Current week metrics:
{{current_metrics_json}}

Previous week metrics:
{{previous_metrics_json}}

Monthly targets:
{{monthly_targets_json}}
```

```
Return JSON:
{
  "headline": "one-line summary of the week (positive or concerning)",
  "executive_summary": "3-4 sentences for busy stakeholders",
  "wins": ["top 3 positive trends with specific numbers"],
  "concerns": ["areas that need attention with specific numbers"],
  "week_over_week": [
    {"metric": "...", "change": "+/-X%", "trend": "up|down|flat", "context":
"why this matters"}
  ],
  "on_track_for_monthly": true/false,
  "gap_to_target": "what needs to happen to hit monthly goals",
  "recommended_focus": ["top 3 priorities for next week"]
}
```

PROMPT 20: MULTI-LANGUAGE RESPONSE TRANSLATOR International

```
Detect the language of this customer message and generate a response in the
same language.

Customer message:
{{customer_message}}

Response template (English):
{{english_response_template}}

Business context: {{context}}

Return JSON:
{
  "detected_language": "ISO 639-1 code",
  "detected_language_name": "English name of language",
  "confidence": 0.0-1.0,
  "translated_response": "the response in the detected language",
  "cultural_notes": "any cultural considerations for this market",
  "formality_adjusted": true/false,
```

```
  "formality_note": "explanation of formality adjustments if any"
}
```

Prompt Engineering Best Practices for Automation

1. **Always request JSON output.** Specify the exact schema. Use OpenAI's structured output mode when available.
2. **Include edge case instructions.** "If the data is missing, return null for that field. Never fabricate information."
3. **Provide examples when the task is ambiguous.** One-shot or few-shot examples dramatically improve consistency.
4. **Set the context explicitly.** Tell the AI who it is, what the data represents, and how the output will be used.
5. **Use temperature 0 for classification/extraction** and temperature 0.7-0.9 for creative content generation.
6. **Validate AI output in your automation.** After the AI module, add a JSON parse step and check for required fields before proceeding.
7. **Version your prompts.** Store them in a data store or config table. When you improve a prompt, you want to know which version generated each output.
8. **Monitor output quality.** Periodically sample AI outputs and rate them. If quality degrades, investigate prompt drift or model updates.

JEDAI FLOW

The AI Automation Mastery Guide - Advanced Edition 2026

jedaiflow.com • AI-Powered Automation for Modern Business

Copyright © 2026 Jedai Flow. All rights reserved. This material is for the exclusive use of the purchaser. Do not distribute.